The Algorithm

The complete code behind the portfolio comparison

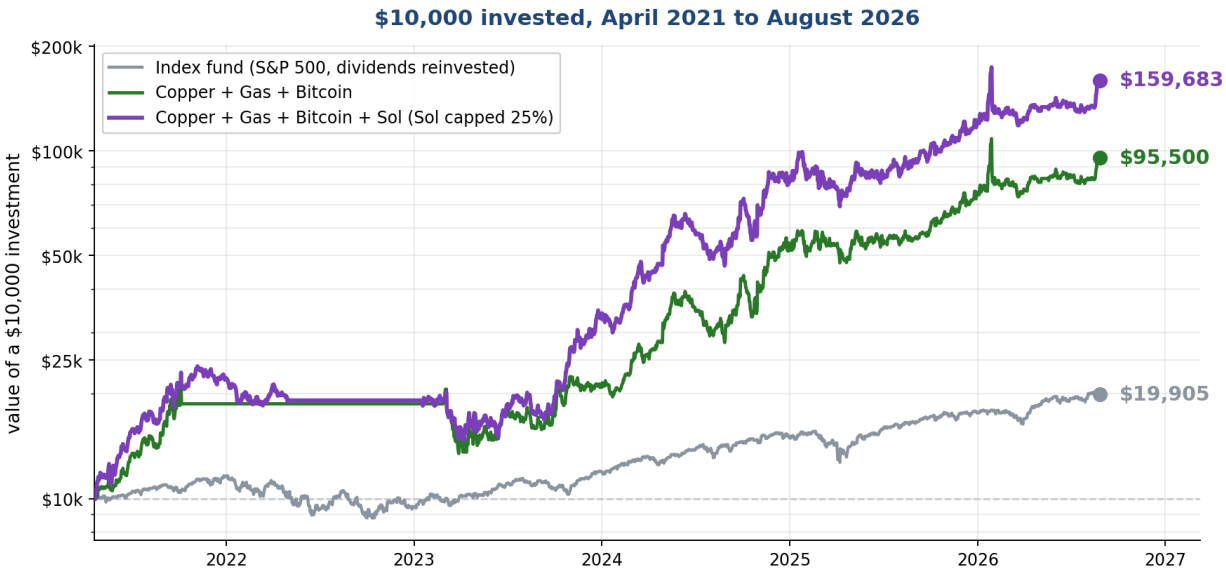
Robin Langer · August 2026

What this document is

One Python file — `make_figure.py`, 247 lines — that goes from nothing to the finished diagram: it downloads prices from Yahoo Finance, computes the momentum signal for all four assets, runs the capital-recycling rule with the Solana cap, and draws the chart.

It is **self-contained**. It imports no other file in the project. Copy it onto a bare machine with `numpy`, `pandas`, `matplotlib` and `yfinance` installed and it reproduces the diagram from scratch.

1. What it produces



Running it

```
pip install numpy pandas matplotlib yfinance

python3 make_figure.py # writes three_way.png
python3 make_figure.py --cap 0.20 # a different Solana cap
python3 make_figure.py --capital 25000 # a different starting sum
```

Actual output, run 27 August 2026

```
Fetching prices and computing momentum (this takes a minute)...

2021-04-19 to 2026-08-26 ($10,000 invested)

  Index fund (S&P 500 total return) $ 19,905 +99% CAGR 13.7% worst drop -24.5%
  Copper + Gas + Bitcoin $ 95,500 +855% CAGR 52.4% worst drop -35.3%
```

```
 + Sol, capped at 25% $ 159,683 +1497% CAGR 67.8% worst drop -38.7%

Wrote three_way.png
```

2. How it works, in six steps

1. **Fetch** (`fetch`). Daily closes and volumes from Yahoo Finance for `HG=F` (copper), `NG=F` (gas), `BTC-USD` and `SOL-USD`, plus `^SP500TR` for the benchmark. Timezones are stripped and duplicate dates dropped, so the four series can be aligned.

2. **Momentum** (`rolling_ratio`). For each day, weight every daily return in the lookback window by that day's volume relative to the window average, then compound those flows into a pool seeded at the price at the window's start. The pool's final value divided by today's price is the *momentum ratio*. Above 1, the weight of money supports a higher price.

3. **Weights** (`weights_for`). Split the money equally between whichever assets have a ratio above 1, then cap Solana at 25% and push the excess onto the other active assets — or leave it in cash if there are none.

4. **Align** (`build_panel`). Crypto trades seven days a week, futures do not. The crypto calendar is treated as master and futures closes forward-filled onto non-trading days, so a held position persists across a weekend at zero return.

5. **Run** (`run`). Multiply weights by returns, sum across assets, compound. Called twice: once with `cap=0`, which excludes Solana entirely and gives the three-asset portfolio, and once with `cap=0.25`.

6. **Draw**. Three lines on a logarithmic axis, each labelled with its final value.

3. Three details that matter

The one-day lag

`build_panel` ends each signal with `.shift(1)`. Without it the backtest would buy using a closing price it could not have known at the time — the single most common way a backtest invents returns that were never available. One line, and it is the difference between a simulation and a fantasy.

The benchmark includes dividends

`BENCHMARK = "^SP500TR"` is the S&P 500 *total return* index, with dividends reinvested. The plain price index (`^GSPC`) would have returned +107% over this period instead of +125% — so using it would have flattered the strategy by about 18 points it has not earned. If you are going to lose to a benchmark, lose to the honest one.

cap=0 is not a special case

The three-asset portfolio is not separate code. It is the same function called with `cap=0.0`, which forces Solana's weight to zero and redistributes it. That guarantees the two lines on the chart differ by *exactly one parameter* — so the gap between them is the effect of adding Solana, and nothing else.

4. The complete listing

```
"""
Generate the copper-gas-bitcoin-sol comparison diagram, from scratch.

Self-contained: fetches prices from yfinance, computes the bookmaker-pool
momentum signal for each asset, applies the capital-recycling rule with
the 25% Solana cap, and plots the result against an index fund.

    python3 make_figure.py # writes three_way.png
    python3 make_figure.py --cap 0.20 # try a different Solana cap
    python3 make_figure.py --capital 25000

Requires: numpy pandas matplotlib yfinance
"""

import argparse
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# -- What we trade, and how the signal is computed for each -----------
# lookback is the per-asset optimum from the cross-asset study:
# 180 days for everything except natural gas, where 90 works better.

ASSETS = {
    "COPPER": {"symbol": "HG=F", "lookback": 180},
    "NATGAS": {"symbol": "NG=F", "lookback": 90},
    "BTC": {"symbol": "BTC-USD", "lookback": 180},
    "SOL": {"symbol": "SOL-USD", "lookback": 180},
}
ORDER = ["COPPER", "NATGAS", "BTC", "SOL"]

BENCHMARK = "^SP500TR" # S&P 500 TOTAL return: dividends reinvested.
                       # The honest benchmark - a price-only index
                       # would understate it by ~2% a year.

START = "2021-04-19" # first date on which all four have a full
                       # lookback window behind them
CAPITAL = 10_000.0
SOL_CAP = 0.25

ACCENT, GREEN, GREY, PURPLE = "#1e4678", "#287828", "#8a94a0", "#7a3fb8"


# -- 1. Prices --------------------------------------------------------

def fetch(symbol, start="2020-01-01"):
    """Daily closes and volumes, tz-stripped and de-duplicated."""
    h = yf.Ticker(symbol).history(start=start, interval="1d",
                                  auto_adjust=False)
    h = h.dropna(subset=["Close"])
    h.index = pd.to_datetime(h.index, utc=True).tz_localize(None).normalize()
    return h[~h.index.duplicated(keep="last")]


# -- 2. The momentum signal (the "bookmaker pool") --------------------

def rolling_ratio(hist, lookback):
    """
    Momentum ratio for every day with a full lookback window.

    Each day's net flow is (volume / mean volume) * return: a price move
    counts for more when more money stood behind it. Those flows are
    compounded into a pool seeded at the price at the start of the
    window. The pool's final value is the 'momentum price'; we return
    its ratio to the actual close.

        ratio > 1 -> the weight of money supports a higher price
        ratio <= 1 -> money is leaving; stand aside
    """
```

```
    closes = hist["Close"].values.astype(float)
    volumes = hist["Volume"].values.astype(float)
    out = np.full(len(closes), np.nan)

    for i in range(lookback, len(closes)):
        window_close = closes[i - lookback:i + 1]
        window_vol = volumes[i - lookback:i + 1]

        returns = np.diff(window_close) / window_close[:-1]
        vols = window_vol[1:]

        mean_vol = vols.mean()
        if mean_vol == 0 or not np.isfinite(mean_vol):
            continue
        net_flows = (vols / mean_vol) * returns

        pool = float(window_close[0])
        momentum_price = pool
        for flow in net_flows:
            pool += flow * momentum_price
            momentum_price = pool

        if np.isfinite(momentum_price) and closes[i] != 0:
            out[i] = momentum_price / closes[i]

    return pd.Series(out, index=hist.index)


# -- 3. Capital recycling, with the Solana cap ------------------------

def weights_for(active, cap):
    """
    Equal weight across every asset whose signal is on, except that
    Solana is never allowed more than `cap`. The excess goes to the
    other active assets, or stays in cash if there are none.

    active: {asset: bool}. Returns {asset: weight}; the shortfall
    below 1.0 is cash.
    """
    on = [a for a in ORDER if active[a]]
    w = {a: 0.0 for a in ORDER}
    if not on:
        return w # everything in cash

    for a in on:
        w[a] = 1.0 / len(on)

    if cap is not None and w["SOL"] > cap:
        excess = w["SOL"] - cap
        w["SOL"] = cap
        others = [a for a in on if a != "SOL"]
        for a in others: # empty => stays cash
            w[a] += excess / len(others)

    return w


# -- 4. Build the daily panel -----------------------------------------

def build_panel(start=START):
    """
    Returns (returns, signals), both indexed by the 7-day crypto
    calendar.

    Crypto trades every day; futures do not. We take the crypto
    calendar as master and forward-fill futures closes onto non-trading
    days, so a held position simply persists over the weekend with zero
    return. Signals persist the same way.
    """
    hist, ratio = {}, {}
    for name in ORDER:
        h = fetch(ASSETS[name]["symbol"])
        hist[name] = h
        ratio[name] = rolling_ratio(h, ASSETS[name]["lookback"])

```

```
    calendar = hist["BTC"].index
    calendar = calendar[calendar >= pd.Timestamp(start)]

    returns, signals = {}, {}
    for name in ORDER:
        close = hist[name]["Close"].reindex(calendar).ffill()
        returns[name] = close.pct_change().fillna(0.0)

        s = (ratio[name] > 1.0).where(ratio[name].notna())
        s = s.reindex(calendar).ffill().fillna(0.0).astype(bool)
        signals[name] = s.shift(1).fillna(False).astype(bool)
        # shift(1): you cannot trade on a close you have not seen yet

    return pd.DataFrame(returns)[ORDER], pd.DataFrame(signals)[ORDER]


# -- 5. Run the portfolio ---------------------------------------------

def run(returns, signals, cap, capital=CAPITAL):
    """Daily NAV for one choice of Solana cap. cap=0 excludes Solana."""
    W = pd.DataFrame([weights_for(signals.loc[d], cap) for d in signals.index],
                     index=signals.index, columns=ORDER)
    daily = (W * returns).sum(axis=1)
    return capital * (1 + daily).cumprod(), W


def summarise(label, nav, capital=CAPITAL):
    years = (nav.index[-1] - nav.index[0]).days / 365.25
    growth = nav.iloc[-1] / capital
    drawdown = (nav / nav.cummax() - 1).min()
    print(f" {label:34s} ${nav.iloc[-1]:>10,.0f} "
          f"{(growth - 1) * 100:>+8.0f}% "
          f"CAGR {(growth ** (1 / years) - 1) * 100:>5.1f}% "
          f"worst drop {drawdown * 100:>6.1f}%")


# -- 6. The diagram ---------------------------------------------------

def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--cap", type=float, default=SOL_CAP,
                    help="Solana cap as a fraction. Default: 0.25")
    ap.add_argument("--capital", type=float, default=CAPITAL)
    ap.add_argument("--out", default="three_way.png")
    args = ap.parse_args()

    print("Fetching prices and computing momentum (this takes a minute)...")
    returns, signals = build_panel()

    nav_three, _ = run(returns, signals, cap=0.0, capital=args.capital)
    nav_four, _ = run(returns, signals, cap=args.cap, capital=args.capital)
    start, end = nav_three.index[0], nav_three.index[-1]

    # Benchmark, rebased to the same starting capital on the same day.
    bench = fetch(BENCHMARK, start=str(start.date()))["Close"]
    bench = bench.reindex(nav_three.index).ffill().bfill()
    nav_index = args.capital * bench / bench.iloc[0]

    print(f"\n{start.date()} to {end.date()} "
          f"(${args.capital:,.0f} invested)\n")
    summarise("Index fund (S&P 500 total return)", nav_index, args.capital)
    summarise("Copper + Gas + Bitcoin", nav_three, args.capital)
    summarise(f"+ Sol, capped at {args.cap * 100:.0f}%", nav_four, args.capital)

    fig, ax = plt.subplots(figsize=(10.6, 5.0))

    series = [
        (nav_index, GREY, "Index fund (S&P 500, dividends reinvested)", 1.9),
        (nav_three, GREEN, "Copper + Gas + Bitcoin", 2.0),
        (nav_four, PURPLE,
         f"Copper + Gas + Bitcoin + Sol (Sol capped {args.cap * 100:.0f}%)", 2.4),
    ]
    for nav, colour, label, width in series:
        ax.plot(nav.index, nav.values, color=colour, lw=width, label=label, zorder=3)
        ax.plot(nav.index[-1], nav.iloc[-1], "o", color=colour, ms=8, zorder=4)
```

```
        ax.annotate(f"${nav.iloc[-1]:,.0f}", xy=(nav.index[-1], nav.iloc[-1]),
                    xytext=(12, 0), textcoords="offset points", color=colour,
                    fontsize=12, fontweight="bold", va="center")

    ax.axhline(args.capital, color="#c8ced6", ls="--", lw=1.1, zorder=1)
    ax.set_yscale("log")
    ax.set_yticks([10000, 25000, 50000, 100000, 200000])
    ax.set_yticklabels(["$10k", "$25k", "$50k", "$100k", "$200k"])
    ax.set_xlim(start, end + (end - start) * 0.10)
    ax.set_ylabel(f"value of a ${args.capital:,.0f} investment", fontsize=11)
    ax.grid(alpha=0.25, which="both")
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
    ax.legend(loc="upper left", fontsize=10, framealpha=0.95)
    ax.set_title(f"${args.capital:,.0f} invested, "
                 f"{start:%B %Y} to {end:%B %Y}",
                 color=ACCENT, fontsize=13, fontweight="bold", pad=12)

    fig.tight_layout()
    fig.savefig(args.out, dpi=170, bbox_inches="tight", facecolor="white")
    print(f"\nWrote {args.out}")


if __name__ == "__main__":
    main()
```

Verification

The momentum arithmetic in `rolling_ratio` was checked against the original `momentum_single.py` on identical data: **maximum difference 0.000e+00 across 1,584 days, and 100% agreement on the hold/cash signal.** It is the same algorithm, not a paraphrase of it.

Historical returns do not guarantee future returns.

This code simulates trades at daily closing prices with no fees, spread, slippage or tax. Live results will be lower.